

ABC Trading Java Rewrite

ABC Trading Java Rewrite

Implementation Status Report

Scope: java/src/main/java, java/src/test/java, the Python JPyPe facade, and the Rust-vs-Java reconciliation tools.

Purpose: describe what is implemented in Java so far, how the pieces fit together, how each class is used, and why the boundaries were chosen.

Status: implementation snapshot as of 2026-08-23.

1. Executive Summary

The Java rewrite is a deterministic, synchronous, backtest-oriented trading runtime shaped after Nautilus Trader. Java owns the runtime loop, message routing, order state, risk boundary, simulated execution, portfolio position state, and event logging. Python owns the strategy callback and historical-data loading at the current integration boundary.

The strongest verified feature is cross-backend behavioral reconciliation. The shared historical-bar workflow currently compares 228 semantic lifecycle rows between Nautilus and Java. The dedicated L3 MBO fixture compares two fills across a named venue order, queue-ahead trades, maker/taker classification, and deterministic fill ordering.

Verified results

```
mvn -q clean test
MATCH rows=228
MATCH L3 fills=2
```

The current Java test suite covers the deterministic runtime, L3 queue behavior, and account settlement. Account-state events expose total, locked, free, initial-margin, maintenance-margin, and currency information.

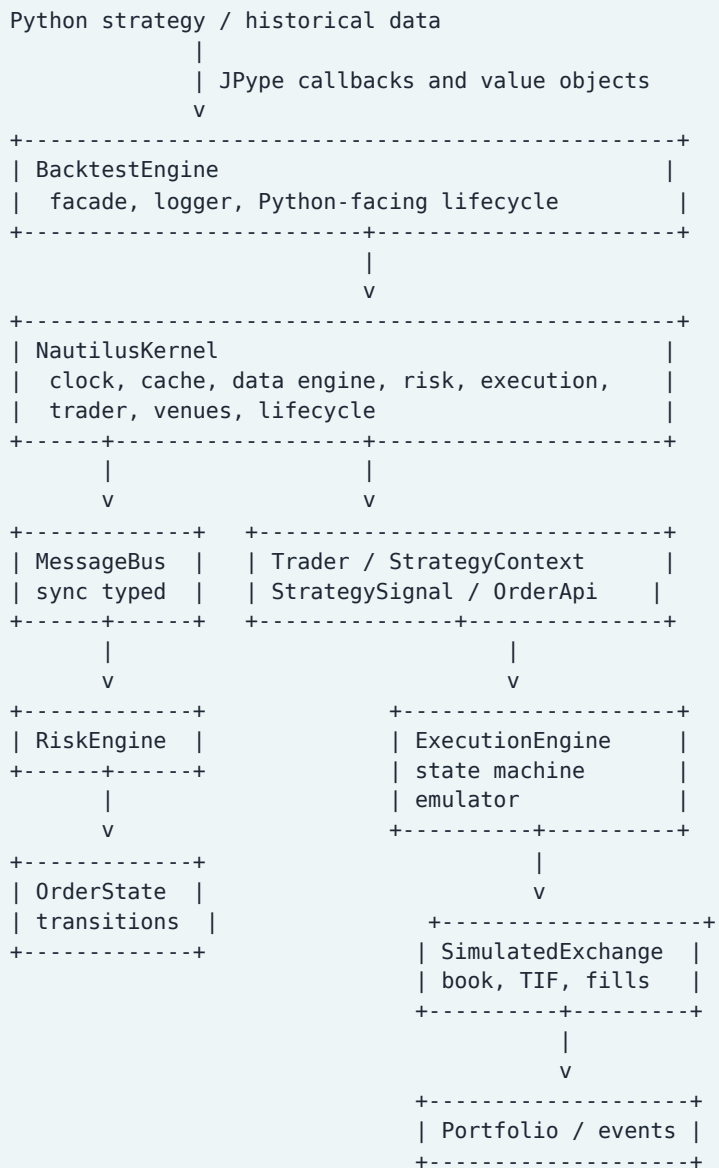
Main implemented feature groups

- synchronous typed message bus with topic routing, priorities, wildcards, endpoints, and correlation callbacks
- component lifecycle and kernel composition
- Python strategy callbacks through JPyPe
- market and limit orders
- stop-market and stop-limit orders
- trailing stop market and trailing stop limit orders
- order lifecycle state machine
- cancel, modify, reject, deny, expiry, IOC, FOK, GTD, and DAY behavior
- local order emulation
- fixed tick-size configuration for Rust-supported TICKS trailing offsets
- L2 aggregate order-book snapshots and deltas
- L3 individual venue-order snapshots and deltas
- trade-driven L3 queue-ahead consumption

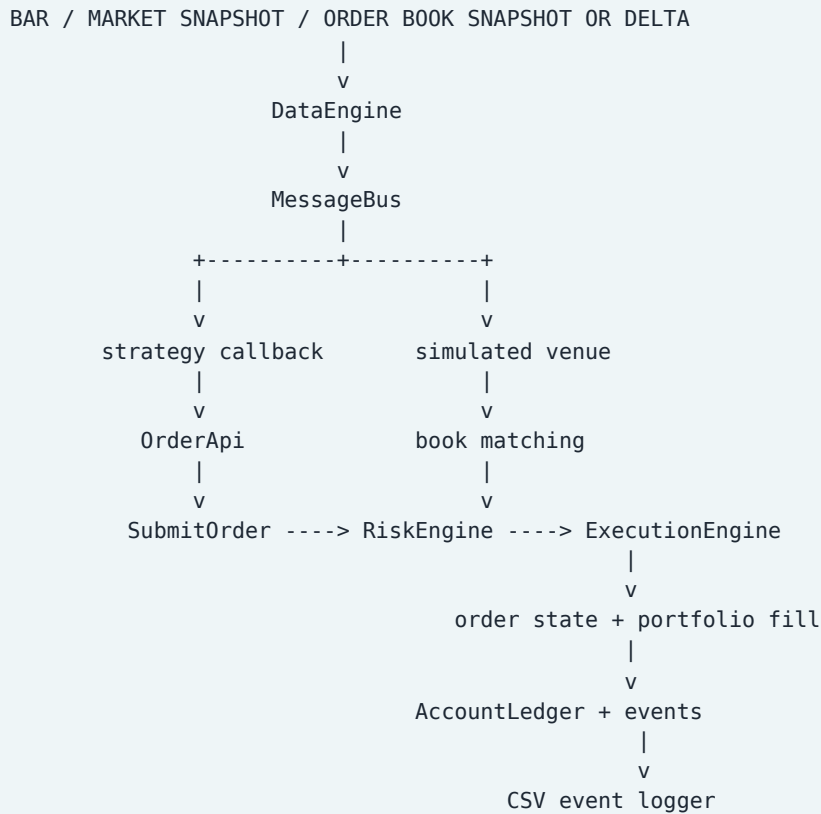
- partial fills and multiple fills across price levels
- maker/taker liquidity classification
- static operation latency
- multiple fee models
- cash and margin account settlement
- instrument-specific margin rates and explicit FX conversion
- multi-currency balance views and account-state events
- CSV lifecycle logging
- Rust-vs-Java reconciliation fixtures and comparators

2. Architecture

2.1 Runtime composition



2.2 Data and order flow



2.3 Design principles

1. **Synchronous by default.** The core MessageBus calls handlers during publish; it is not a worker queue.
2. **Explicit boundaries.** Data, risk, execution, portfolio, strategy, and reconciliation are separate Java packages.
3. **Deterministic ordering.** Input bars are sorted by timestamp and symbol. Bus handlers use priority and registration order. Working orders use price priority and insertion sequence.
4. **Immutable messages.** Records are used for commands, events, market snapshots, book levels, and order state.
5. **Primitive hot path.** Prices, quantities, timestamps, and PnL are currently primitive-backed double/int/long. Exact decimal behavior is established through shared precision and tolerance rules rather than BigDecimal in the hot path.
6. **Source anchoring.** Rust paths are recorded in capability metadata and the reconciliation harness compares observable behavior rather than only totals.

3. Package Structure

```
com.abc.trading
|-- backtest      runner facade, iterator, capability metadata
|-- cache         instruments, positions, orders
|-- common.factories typed order construction
|-- data          bars, market snapshots, book snapshots/deltas, ticks
|-- events        event schema and CSV logger
|-- execution     commands, state machine, venue, fills, fees, latency
|-- indicators    EMA and SMA
|-- model         money and immutable order models
|-- msgbus        typed synchronous bus and transport backings
|-- portfolio     positions, balances, margin, PnL, account events
|-- reconciliation Java-side comparison DTOs
|-- risk          risk decisions and trading state
|-- system       kernel, clock, trader, component lifecycle
`-- trading       actors, strategies, context, order API
```

The Python bridge lives under `python/abc_trading`; reconciliation scripts live under `recon`.

4. Feature Matrix

Feature	Status	Implementation surface	Notes
Typed synchronous message bus	Implemented	<code>msgbus.MessageBus</code> , <code>TypedTopicRouter</code> , <code>TypedEndpointMap</code>	Direct dispatch, wildcard topics, priorities, endpoint sends, request/response correlation
Kernel lifecycle	Implemented	<code>NautilusKernel</code> , <code>ComponentLifecycle</code>	Initialization, start, stop, reset, dispose, fault/degrade vocabulary
Strategy callbacks	Implemented	<code>Trader</code> , <code>StrategyContext</code> , <code>Python BacktestEngine</code>	Python strategy receives Java-driven bars and context
Market orders	Implemented	<code>OrderApi</code> , <code>ExecutionEngine</code> , <code>SimulatedExchange</code>	Immediate or latency-delayed execution
Limit orders	Implemented	same	Cross book levels or rest as maker
Stop market/limit	Implemented	order models, trigger evaluation, state machine	Stop-market fills immediately; stop-limit enters TRIGGERED
Trailing stops	Implemented	trailing models, exchange ratchet	PRICE, BASIS_POINTS, TICKS; PRICE_TIER intentionally rejected to match current Rust
Cancel/modify	Implemented	command records, execution client, state machine	Explicit pending and reject states
TIF	Implemented	<code>TimeInForce</code> , exchange	GTC, IOC, FOK, GTD, DAY
Partial/multiple fills	Implemented	working orders and <code>OrderStateMachine</code>	Quantity and average fill price tracked
L2 book snapshots	Implemented	<code>OrderBookSnapshot</code> , <code>BookLevel</code>	Aggregate depth per price level
L2 book deltas	Implemented	<code>OrderBookDelta</code> , <code>BookAction</code>	ADD, UPDATE, DELETE, CLEAR

Feature	Status	Implementation surface	Notes
L3 individual venue queue	Implemented	VenueOrder, OrderBookL3Snapshot, OrderBookL3Delta, TradeTick, SimulatedExchange	Individual order identity, price-time priority, queue-ahead deltas, and trade-driven queue consumption
Liquidity classification	Implemented	OrderFill.liquiditySide	MAKER/TAKER included in reconciliation CSV
Fees	Implemented	FeeModel implementations	Fixed, maker/taker, per-contract, probability, capped, notional/tiered
Static latency	Implemented	LatencyModel, StaticLatencyModel	Operation latency with deterministic timestamp ordering
Position/PnL accounting	Implemented	Portfolio, PositionUpdate, Money	Net position, average price, realized PnL, and commission effects
Cash/margin account	Implemented baseline	AccountLedger, AccountState, AccountBalance, AccountType	Reservations, free/locked balances, cash settlement, initial/maintenance margin, and threshold events
Instrument margin metadata	Implemented	InstrumentSpec, Cache	Quote/base currencies and instrument-specific margin rates
FX conversion	Implemented	FxRateUpdate, AccountLedger, Portfolio, BacktestEngine	Replayable market-data rates for cross-currency margin and PnL conversion
Account-state events	Implemented	AccountStateEvent, Event, CsvEventLogger	Balance and margin transitions are observable in canonical logs
Mark-to-market valuation	Implemented baseline	Portfolio, AccountLedger, MarketDataSnapshot	Mark updates recalculate signed unrealized PnL and threshold flags
Margin model	Implemented baseline	MarginModelType, InstrumentSpec, AccountLedger	Notional-rate and fixed-per-unit formulas with leverage
Decimal accounting	Partial	primitive double	BigDecimal/fixed-point remains a future accounting hardening step
Local order emulator	Implemented	OrderEmulator	Snapshot-triggered local ownership and release
Disruptor bus	Scaffold only	DisruptorMessageBus	Publish method is still a TODO
External ring-buffer backing	Implemented as backing	RingBufferMessageBusBacking	Bounded queue; full-buffer policy currently drops and reports
Persistence/event store	Not implemented	no event-store module	CSV logging is available
Live adapters	Not implemented	no REST/WebSocket adapter layer	Current target is deterministic backtest behavior

5. Class and Type Catalog

The following catalog covers the Java production types currently under `java/src/main/java`.

5.1 Backtest package

Type	Usage	Design rationale
BacktestEngine	Public JPytype-friendly facade; registers venues/instruments/strategies and runs data	Keeps Python integration simple while Java owns orchestration and logging
BacktestDataIterator	Stores and iterates historical bars	Mirrors a deterministic replay boundary and supports reset/clear
BacktestEngineConfig	Immutable runner options	Makes sort/stream/analysis intent explicit even where the current facade uses defaults
BacktestResult	Immutable run summary DTO	Separates result reporting from runtime state
EngineCapability	One capability/status record	Makes Rust mapping and implementation status inspectable
EngineCapabilities	Registry of current capability mappings	Documents implemented and pending Rust-aligned surfaces
SimulatedVenueConfig	Venue flags, latency, and fee configuration	Keeps venue behavior configurable without changing kernel composition

5.2 Cache and common factory packages

Type	Usage	Design rationale
Cache	Instrument venue mapping, positions, and recorded orders	Central deterministic state store shared by risk, execution, and portfolio
OrderFactory	Creates client IDs and typed market/limit/stop/trailing orders	Centralizes identity generation and model construction

5.3 Data package

Type	Usage	Design rationale
Bar	Immutable close-price input for strategy and legacy replay	Minimal bar boundary with primitive fields and stable sequence
MarketDataSnapshot	Carries bid, ask, last, mark, index	Gives trigger evaluation one deterministic multi-source input
BookLevel	Price and aggregate quantity at one level	Small immutable unit for L2 depth
OrderBookSnapshot	Complete immutable bid/ask ladder	Snapshot replacement is simple and deterministic for replay
OrderBookDelta	One book mutation	Mirrors Rust add/update/delete/clear event semantics
VenueOrder	Individual venue order with side, price, size, and sequence	Preserves L3 MBO identity instead of flattening depth into aggregate levels
OrderBookL3Snapshot	Complete individual-order bid/ask state	Establishes deterministic price-time priority at snapshot boundaries
OrderBookL3Delta	Add, update, delete, or clear one venue order	Allows queue position to respond to exact venue-order mutations
TradeTick	Executed market trade with aggressor side	Drives queue-ahead consumption independently from book-depth updates
AggressorSide	Buyer, seller, or no-aggressor trade classification	Determines which passive queue can advance

Type	Usage	Design rationale
FxRateUpdate	Replayable FX conversion-rate input	Makes cross-currency account valuation deterministic and time-ordered
MarginModelType	Notional-rate or fixed-per-unit formula selection	Keeps instrument margin policy explicit rather than hidden in risk code
InstrumentSpec	Base/quote currencies and initial/maintenance margin rates	Carries the instrument facts needed by accounting and risk
BookAction	ADD, UPDATE, DELETE, CLEAR	Makes book mutation intent explicit
TickScheme	Instrument-owned fixed tick increment	Rust currently supports TICKS through price_increment; Java avoids a hard-coded exchange tick in that path
DataEngine	Publishes bars, market snapshots, books, and deltas	Separates data distribution from data ownership
DataClient	Contract for data clients	Extension point for live or external data sources
DataEngineConfig	Immutable data-engine options	Configuration boundary for future subscription/replay policies

5.4 Events package

Type	Usage	Design rationale
Event	Canonical CSV row with lifecycle, order, PnL, commission, liquidity, balance, and margin	One stable schema supports cross-backend comparison and account evidence
EventType	Semantic event vocabulary	Keeps logger output independent from concrete Java event class names
EventLogger	Event logging contract	Allows CSV or another sink later
CsvEventLogger	Writes synchronized CSV rows	Human-readable evidence and simple reconciliation input

5.5 Execution commands and order intents

Type	Usage	Design rationale
SubmitOrder	Rust-shaped command envelope	Carries order model, type, TIF, trigger, trailing, and emulation metadata
CancelOrder	Cancel command	Separates command from resulting canceled event
ModifyOrder	Quantity, price, and trigger modification command	Mirrors Rust optional modify fields
OrderType	Market, limit, stop, and trailing variants	Explicit dispatch rather than implicit model inspection
OrderIntent	Market-style internal execution input	Lightweight transport from risk to venue
LimitOrderIntent	Limit-style internal execution input	Keeps limit price and trailing-limit data explicit
SignalDirection	BUY, SELL, HOLD	Shared side vocabulary for strategies, orders, and events
TimeInForce	GTC, IOC, FOK, GTD, DAY	Controls working-order lifetime and remainder handling
TriggerType	Trigger price source vocabulary	Matches Rust last, quote, mark, index, midpoint, and double-match concepts

Type	Usage	Design rationale
TrailingOffsetType	PRICE, BASIS_POINTS, TICKS, PRICE_TIER	Java rejects PRICE_TIER because current Rust rejects it
OrderAccepted	Accepted market-style event	Makes venue acknowledgement observable
LimitOrderAccepted	Accepted limit-style event	Preserves order-type-specific event flow
OrderDenied	Risk/system denial event	Represents failure before venue acceptance
LimitOrderDenied	Limit risk denial event	Keeps limit denial payload typed
OrderRejected	Venue rejection event	Distinguishes venue rejection from local denial
LimitOrderRejected	Limit venue rejection event	Typed limit rejection counterpart
OrderCanceled	Successful cancel event	Drives terminal CANCELED transition
OrderCancelRejected	Failed cancel event	Restores the prior open state
OrderModified	Successful modify event	Makes accepted updates observable
OrderModifyRejected	Failed modify event	Restores the prior open state
OrderExpired	GTD/DAY expiry event	Drives terminal EXPIRED transition
OrderTriggered	Stop-limit trigger event	Drives ACCEPTED -> TRIGGERED
OrderEmulated	Local emulator ownership event	Drives INITIALIZED -> EMULATED
OrderReleased	Emulator release event	Drives EMULATED -> RELEASED
OrderVoided	Fill-correction event	Represents the Rust terminal correction vocabulary
OrderFill	One execution fill	Carries quantity, price, commission, and liquidity side
SettledOrderFill	Fill after portfolio settlement	Separates raw venue fill from state-settled fill
OrderState	Immutable submitted/filled/remaining snapshot	Makes state queryable without exposing mutable order internals
OrderStatus	Full Rust status vocabulary	Predicates identify open and terminal states
OrderStateMachine	Validates every order transition	Prevents invalid lifecycle jumps and preserves prior pending state
OrderMatchingEngine	Stateless legacy single-price matcher	Retained as a small model-level utility; SimulatedExchange owns current book matching
ExecutionClient	Venue execution contract	Allows simulated and future live clients to share the boundary
BacktestExecutionClient	Adapts SimulatedExchange to ExecutionClient	Keeps execution-engine venue routing independent from simulator details
ExecutionEngine	Risk, emulator, client routing, state, and portfolio coordination	Central execution boundary corresponding to Rust execution orchestration
SimulatedExchange	Working orders, book consumption, triggers, TIF, latency, fills	Owns venue-specific matching and deterministic liquidity consumption
OrderEmulator	Holds locally emulated submit commands and releases on snapshots	Mirrors Rust emulator ownership before venue submission
Commission	Immutable fee amount/currency	Keeps fill fee data explicit and testable

Type	Usage	Design rationale
FeeModel	Fee calculation contract	Supports interchangeable venue fee policies
FixedFeeModel	Fixed commission policy	Useful for per-order or one-time fees
MakerTakerFeeModel	Rate-based maker/taker policy	Calculates fee from notional and liquidity side
PerContractFeeModel	Quantity-based policy	Models contract-style fees without notional math
ProbabilityPriceFeeModel	Probability/price fee policy	Supports specialized instrument fee formulas
CappedOptionFeeModel	Capped option fee policy	Prevents fee growth beyond a configured cap
TieredNotionalOptionFeeModel	Notional-tier option fee policy	Models tiered fee schedules
LiquiditySide	MAKER or TAKER	Explicitly records how liquidity was consumed or provided
LatencyModel	Operation latency contract	Allows deterministic or future stochastic latency models
StaticLatencyModel	Base plus operation-specific latency	Current reproducible latency implementation
VenueId	Validated venue value object	Rust uses validated identity strings rather than enums
DeterministicOrderId	Deterministic order identity helper	Supports reproducible event correlation
ExecutionEngineConfig	Immutable execution configuration DTO	Future configuration hook for execution policy

5.6 Order model package

Type	Usage	Design rationale
Order	Sealed common order interface	Gives factories and submit commands one typed model boundary
MarketOrder	Immediate market order model	Immutable validated order metadata
LimitOrder	Resting/crossing limit order model	Separates limit price from market execution
StopMarketOrder	Triggered market order model	Carries trigger price and trigger source
StopLimitOrder	Triggered limit order model	Carries both trigger and limit price
TrailingStopMarketOrder	Dynamic stop-market model	Carries activation and offset metadata
TrailingStopLimitOrder	Dynamic stop-limit model	Adds limit offset to trailing stop metadata
Money	Immutable amount/currency value	Shared primitive-backed monetary boundary; full account precision remains future work

5.7 Indicators package

Type	Usage	Design rationale
ExponentialMovingAverage	Stateful EMA updates	Keeps indicator state in Java and minimizes bridge calls
SimpleMovingAverage	Stateful rolling average	Matches strategy warm-up behavior with explicit count/initialization

5.8 Message bus package

Type	Usage	Design rationale
MessageBus	Typed synchronous publish/send/request/response	Core in-memory runtime path; no queue by default
TypedTopicRouter	Wildcard topic matching with cached indexes	Separates topic routing from generic bus storage
TypedEndpointMap	Exact typed endpoint lookup	Models point-to-point Rust endpoint delivery
Handler	Generic typed callback	Keeps message bus API type-safe
Serializer	Serialization contract	Boundary for external bus messages
JacksonSerializer	Jackson implementation	Uses existing project dependency for structured payloads
BusMessage	External message envelope	Carries payload type and serialized payload
MessageBusEvent	Topic/payload event record	Data unit for transport-backed bus implementations
MessageBusRouter	Legacy untyped routing contract	Extension point for non-generic bus consumers
MessageHandler	Legacy untyped callback contract	Supports the Disruptor-shaped scaffold
MessageBusBacking	External transport backing contract	Separates transport buffering from synchronous bus semantics
RingBufferMessageBusBacking	Bounded ArrayBlockingQueue backing	External-flow buffering with explicit full-buffer behavior
DisruptorMessageBus	LMAX Disruptor-shaped facade	Performance experiment/scaffold; publish integration remains TODO
SerializationEncoding	Encoding vocabulary	Configuration marker for serialized messages

5.9 Portfolio, risk, and reconciliation packages

Type	Usage	Design rationale
Portfolio	Applies fills, updates positions, tracks average price, PnL, and account settlement	Keeps position and account transitions coordinated after execution
PositionUpdate	Post-fill position/PnL event	Makes portfolio transition observable
PortfolioConfig	PnL/snapshot options	Configuration boundary for future account behavior
AccountLedger	Tracks currency balances, order reservations, position margin, FX conversion, and settlement	Isolates account mutation from order matching and supports deterministic snapshots
AccountState	Immutable primary balance and margin snapshot with per-currency balances	Mirrors Nautilus AccountState as an observable accounting boundary
AccountBalance	Total, locked, and free amount for one currency	Preserves the invariant $\text{total} = \text{locked} + \text{free}$
AccountType	CASH or MARGIN settlement policy	Makes borrowing and position-reservation semantics explicit
AccountStateEvent	Publishes an account snapshot after acceptance or settlement	Allows account changes to enter the same event evidence stream

Type	Usage	Design rationale
AccountMarginCall	Typed notification when equity falls below maintenance margin	Separates a warning threshold from ordinary account snapshots
AccountLiquidationRequired	Typed notification when equity cannot support maintenance	Makes liquidation-required state observable without silently inventing a fill
RiskEngine	Validates side, quantity, instrument, price, notional, trading state, and available margin	Keeps risk before venue execution and prevents unsupported exposure
RiskDecision	Allow/reject result	Separates risk result from order event emission
RiskEngineConfig	Immutable risk options	Future policy configuration boundary
TradingState	Active, halted, reducing state vocabulary	Enables global risk policy changes
ReconciliationComparator	Java-side comparison utility	Keeps semantic comparison logic available inside the Java module
ReconciliationResult	Immutable comparison result	Carries match status, row count, and mismatch information

5.10 System and trading packages

Type	Usage	Design rationale
NautilusKernel	Composition root and chronological replay owner	Mirrors Nautilus runtime orchestration and owns global input sequence
NautilusKernelBuilder	Kernel construction helper	Provides a future fluent composition boundary
NautilusKernelConfig	Kernel options	Separates runtime policy from kernel implementation
Clock	Clock abstraction	Allows simulated and future real clocks
SimulatedClock	Replay clock	Makes event time explicit and testable
ComponentLifecycle	Shared lifecycle transition table	Centralizes valid component state changes
ComponentState	Lifecycle state vocabulary	Mirrors initialized/ready/running/stopped/faulted concepts
ComponentTrigger	Lifecycle transition vocabulary	Keeps trigger names separate from state names
Trader	Strategy registration, contexts, and bar subscriptions	Owns strategy lifecycle and symbol routing
Actor	Actor lifecycle contract	Structural boundary for event-driven components
StrategyHandler	Java callback contract used by Python proxy	Small callback surface for strategy execution
StrategyContext	Current bar, position, sequence, and order API	Gives strategies controlled runtime access
OrderApi	Creates and publishes typed orders; cancel/modify/emulation	Keeps strategy code independent from command construction details
StrategySignal	Strategy-generated signal event	Separates intent observation from order submission
ExecutionAlgorithm	Execution-algorithm contract	Extension point for TWAP/VWAP-like algorithms

6. Important Runtime Details

6.1 Order lifecycle

```
INITIALIZED
|
+--> DENIED
|
+--> EMULATED --> RELEASED --> SUBMITTED --> ACCEPTED
|
|                                     |
|                                     +--> PENDING_UPDATE --> ACCEPTED
|                                     |
|                                     \-> PARTIALLY_FILLED
|                                     +--> PENDING_CANCEL --> CANCELED
|                                     +--> REJECTED
|
+--> PARTIALLY_FILLED --> FILLED
|
|                                     \-> EXPIRED / CANCELED
+--> EXPIRED
+--> CANCELED
+--> FILLED
+--> VOIDED after fill correction
```

Stop-market orders execute immediately on trigger. Stop-limit orders emit TRIGGERED and then match as limit orders. Trailing orders first activate and ratchet their trigger/limit prices before they can trigger.

6.2 Order-book behavior

OrderBookSnapshot replaces the visible L2 book. OrderBookDelta mutates a persistent book:

```
ADD      -> add quantity at price
UPDATE   -> replace quantity at price
DELETE   -> remove price level
CLEAR    -> clear one side
```

Market buys read asks from lowest to highest. Market sells read bids from highest to lowest. Crossed limits take available opposite-side liquidity; unfilled limits rest as makers. Working strategy orders are sorted by better price first and insertion sequence second.

For L3 MBO, each VenueOrder remains distinct. Orders at the same price are ordered by venue sequence and then ID. A resting client order snapshots the quantity and IDs ahead of it. Venue deletes and size decreases advance that queue; size increases retain the venue order's position. A trade tick advances only the passive side selected by its aggressor side, consuming queue-ahead quantity before a client maker order receives a fill.

6.3 Account and margin behavior

AccountLedger is deliberately downstream of execution and upstream of account-state logging:

```
accepted order -> reserve initial margin or cash
venue fill     -> release proportional reservation
               -> apply commission and realized PnL
               -> update position margin
               -> publish AccountStateEvent
cancel/expiry/reject -> release reservation
```

Margin accounts reserve quote-currency notional multiplied by the instrument's initial-margin rate and divided by leverage. Open positions report both initial and maintenance margin using InstrumentSpec. Cash accounts reserve buy-side funds, settle buy/sell notional in the quote currency, and reject uncovered sells. Additional currencies can be deposited and converted through explicitly configured FX rates; missing conversion data rejects a reservation rather than silently treating currencies as equal.

These rules are a deterministic backtest baseline. They do not yet model every Nautilus derivative margin model, liquidation execution policy, or live FX feed.

6.4 Numeric representation

The current runtime uses primitive `double` prices and PnL for bridge and hot-path simplicity. Raw source prices are not rounded in the Python data loader. Reconciliation uses shared instrument precision and a numeric tolerance. A future production accounting layer should use fixed-point integers or `BigDecimal` for money and exact decimal settlement while retaining primitives for routing/indexing where appropriate.

7. Python Integration

The Python facade is intentionally thin:

```
abc_trading.backtest.engine.BacktestEngine
|
+--> starts JVM and loads Java classes
+--> converts Python Bars/Snapshots to Java objects
+--> installs JProxy StrategyHandler
+--> exposes market, limit, stop, trailing, cancel, modify
```

Python strategy callbacks receive a Java-owned `StrategyContext` wrapper. Java increments the global input sequence before publishing each input, so Python-generated signals and Java-generated execution events share the same chronology.

8. Reconciliation Evidence

Existing bar workflow

```
shared immutable OHLCV CSV
|
+--> Java BacktestEngine
|
+--> Nautilus BacktestEngine
|
+--> semantic CSV comparator
```

Compared semantic events:

- `SIGNAL`
- `ORDER_FILL`
- `POSITION_UPDATE`

Backend-only transport events and backend-specific identifiers are excluded intentionally. Prices, PnL, and commissions are compared numerically.

Order-book workflow

Shared fixture:

```
recon/order_book_market_data.csv
```

Both implementations produce:

```
market-buy-1 101.00 x 3  TAKER
market-buy-1 101.01 x 3  TAKER
market-sell-1 99.00 x 5  TAKER
market-sell-1 98.99 x 2  TAKER
limit-buy-1   100.00 x 2  MAKER
```

The order-book comparator checks order, price, quantity, and liquidity side for every fill.

L3 MBO workflow

Shared fixture:

```
recon/l3_mbo_market_data.json
```

The Java runner and Nautilus runner both load the same individual venue orders, submit the same limit and market orders, process the same seller-aggressor trades, and emit compact fill rows. The comparator verifies client order, price, quantity, liquidity side, and fill order:

```
MATCH L3 fills=2
```

Nautilus `venue_order_id` identifies the client order's venue assignment. The Java simulator additionally records the passive L3 book order ID when it is known; that diagnostic identity is intentionally not treated as a Rust parity field because it is not present in Nautilus `OrderFilled`.

Account workflow

Account evidence is emitted through the same Java CSV stream as lifecycle events. `ACCOUNT_STATE` rows contain the primary currency, total, locked and free balances, initial margin, and maintenance margin. The account tests cover margin reservations, cash notional settlement, commissions, realized PnL, instrument-specific rates, FX conversion, uncovered cash sells, and additional currency balances.

9. Tests and Validation

Current Java tests cover:

- message bus routing and ordering
- lifecycle transitions
- kernel sorting and clock behavior
- risk validation
- order API construction
- fee and latency models
- order state transitions
- partial and multiple fills
- cancel, modify, expiry, IOC, FOK, and GTD
- stop and trailing triggers
- emulation release
- tick configuration
- L2 depth traversal and FIFO
- persistent order-book deltas
- individual L3 venue-order identity and queue-ahead changes
- trade-driven queue consumption and aggressor-side behavior
- portfolio position/PnL behavior
- cash/margin account settlement and account-state events

The Java module currently contains 133 production source files and 15 test files. The test suite is intentionally focused on the current deterministic backtest slice; it is not yet a complete replacement of all Nautilus modules.

10. Current Gaps and Recommended Next Work

1. **Exact account parity:** the current baseline needs broader Rust account-event fixtures, full derivative margin models, liquidation execution policy, and market-driven FX coverage.
2. **Persistent event store:** replayable event persistence beyond CSV evidence logs.
3. **Live adapters:** REST/WebSocket data and execution clients, reconnects, throttling, and external reconciliation.
4. **Exact accounting:** fixed-point or BigDecimal money model with explicit currency precision.
5. **Execution algorithms:** actual algorithm scheduling rather than the current interface boundary.
6. **Disruptor integration:** complete the DisruptorMessageBus publication path only if profiling shows the synchronous bus is insufficient.

11. Design Conclusion

The Java implementation has moved beyond isolated skeleton classes. It now forms a coherent deterministic backtest runtime with explicit Rust-shaped ownership boundaries and executable cross-backend evidence. L3 queue semantics and a baseline cash/margin ledger are implemented; the next parity work is exact derivative account behavior and richer market-data-driven valuation.

12. Business Knowledge and Rust Reference

12.1 Why these boundaries matter to a trading system

The rewrite follows trading-domain ownership rather than treating the application as a generic event processor:

Business concept	Operational meaning	Java owner	Nautilus/Rust reference
Market data	The venue's observable state and executed trades	DataEngine, OrderBook*, TradeTick	crates/data, crates/model/src/data
Price-time priority	Earlier and better-priced liquidity receives execution first	SimulatedExchange.L3BookState	crates/model/src/orderbook, matching engine
Queue ahead	A passive order cannot fill until visible liquidity ahead is consumed or removed	WorkingOrder queue fields and L3 state	crates/execution/src/matching_engine/engine.rs queue methods
Maker/taker	Determines execution role and often fee schedule	LiquiditySide, FeeModel	LiquiditySide, fill-model paths in matching engine

Business concept	Operational meaning	Java owner	Nautilus/Rust reference
Risk	Prevents invalid or unaffordable exposure before routing	RiskEngine	crates/risk/src/engine
Free versus locked funds	Separates immediately tradable capital from order/position commitments	AccountLedger, AccountState	AccountBalance, CashAccount, MarginAccount
Initial versus maintenance margin	Initial margin gates new exposure; maintenance margin measures ongoing safety	InstrumentSpec, AccountLedger	MarginBalance, AccountsManager
Realized PnL	PnL becomes final when a position is reduced or closed	Portfolio	AccountsManager::update_balances, position PnL methods
Settlement	A fill changes positions, balances, commissions, and observable state	ExecutionEngine and Portfolio	portfolio event and account-state flow

12.2 Rust source map

The Java implementation is anchored to these Nautilus areas:

Java area	Nautilus reference	Reason for the mapping
MessageBus and topic routing	crates/common/src/message_bus and component messaging	Preserve synchronous in-process dispatch semantics for deterministic replay
NautilusKernel and lifecycle	crates/system, crates/common/src/actor	Keep composition, clock, lifecycle, and ownership explicit
Order models and state machine	crates/model/src/orders, crates/execution/src/engine	Match validated order vocabulary and legal transitions
SimulatedExchange	crates/backtest/src/exchange, crates/execution/src/matching_engine	Keep venue matching, triggers, TIF, fills, and queue behavior together
L3 queue tracking	engine.rs: snapshot_queue_position, decrement_queue_on_trade, advance_l3_queue_on_delete, adjust_l3_queue_on_update	Preserve quantity-ahead and per-book-order behavior
Portfolio and AccountLedger	crates/portfolio/src/portfolio.rs, manager.rs, accounts/cash.rs, accounts/margin.rs	Separate positions, balance settlement, and margin policy
RiskEngine	crates/risk/src/engine/mod.rs	Perform affordability and exposure checks before execution

Java area	Nautilus reference	Reason for the mapping
CSV reconciliation	crates/model events and Python backtest surfaces	Compare observable semantics while excluding backend-only IDs

12.3 Business interpretation of a fill

For a market buy, the simulator consumes the lowest available asks. For a resting buy, a seller-aggressor trade first consumes the bid queue ahead and only the excess can fill the client. A fill then has four consequences:

1. The order state advances by the fill quantity.
2. The portfolio position and average price change.
3. Commission and realized PnL change account equity.
4. Reserved funds and position margin are recalculated and published.

This sequence is why matching, risk, portfolio, and account settlement are separate classes even though one market event can touch all of them.